DAA ASSIGNMENT

K.NAGA GNANESWARA REDDY

CH.SC.U4CSE24219

RED – BLACK TREE

CODE: -

```
#include <stdio.h>
#include <stdlib.h>

#define RED 1
#define BLACK 0

struct Node {
    int data;
    int color;
    struct Node *left, *right, *parent;
};

struct Node *root = NULL;

// Create a new node
struct Node* createNode(int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->color = RED;
    newNode->left = newNode->right = newNode->parent = NULL;
    return newNode;
}

// Left rotation
void rotateLeft(struct Node *x) {
    struct Node *y = x->right;
    x->right = y->left;

    if (y->left != NULL)
```

```
        y->left->parent = x;

    y->parent = x->parent;

    if (x->parent == NULL)
        root = y;
    else if (x == x->parent->left)
        x->parent->left = y;
    else
        x->parent->right = y;

    y->left = x;
    x->parent = y;
}

// Right rotation
void rotateRight(struct Node *x) {
    struct Node *y = x->left;
    x->left = y->right;

    if (y->right != NULL)
        y->right->parent = x;

    y->parent = x->parent;

    if (x->parent == NULL)
        root = y;
    else if (x == x->parent->right)
        x->parent->right = y;
    else
        x->parent->left = y;

    y->right = x;
    x->parent = y;
}

// Fix violations after insertion
void fixViolation(struct Node *z) {
    while (z != root && z->parent->color == RED) {
        struct Node *parent = z->parent;
        struct Node *grandparent = parent->parent;
```

```
        if (parent == grandparent->left) {
            struct Node *uncle = grandparent->right;

            if (uncle && uncle->color == RED) {
                grandparent->color = RED;
                parent->color = BLACK;
                uncle->color = BLACK;
                z = grandparent;
            } else {
                if (z == parent->right) {
                    z = parent;
                    rotateLeft(z);
                }
                parent->color = BLACK;
                grandparent->color = RED;
                rotateRight(grandparent);
            }
        } else {
            struct Node *uncle = grandparent->left;

            if (uncle && uncle->color == RED) {
                grandparent->color = RED;
                parent->color = BLACK;
                uncle->color = BLACK;
                z = grandparent;
            } else {
                if (z == parent->left) {
                    z = parent;
                    rotateRight(z);
                }
                parent->color = BLACK;
                grandparent->color = RED;
                rotateLeft(grandparent);
            }
        }
    }
    root->color = BLACK;
}

// Insert a node
```

```
void insert(int data) {
    struct Node *z = createNode(data);
    struct Node *y = NULL;
    struct Node *x = root;

    while (x != NULL) {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
    }

    z->parent = y;
    if (y == NULL)
        root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;

    fixViolation(z);
}

// Inorder traversal
void inorder(struct Node *root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d(%s) ", root->data, root->color == RED ? "R" : "B");
    inorder(root->right);
}

int main() {
    insert(10);
    insert(20);
    insert(30);
    insert(15);
    insert(25);

    printf("Inorder Traversal of Created Red-Black Tree:\n");
    inorder(root);
```

```
    printf("\n");

    return 0;
}
```

OUTPUT: -

```
┌──(gnaneswar993㉿kali)-[~]
└─$ nano red_black.c

┌──(gnaneswar993㉿kali)-[~]
└─$ gcc red_black.c -o red_black

┌──(gnaneswar993㉿kali)-[~]
└─$ ./red_black
Inorder Traversal of Created Red-Black Tree:
10(B) 15(R) 20(B) 25(R) 30(B)
```

AVL TREE: -

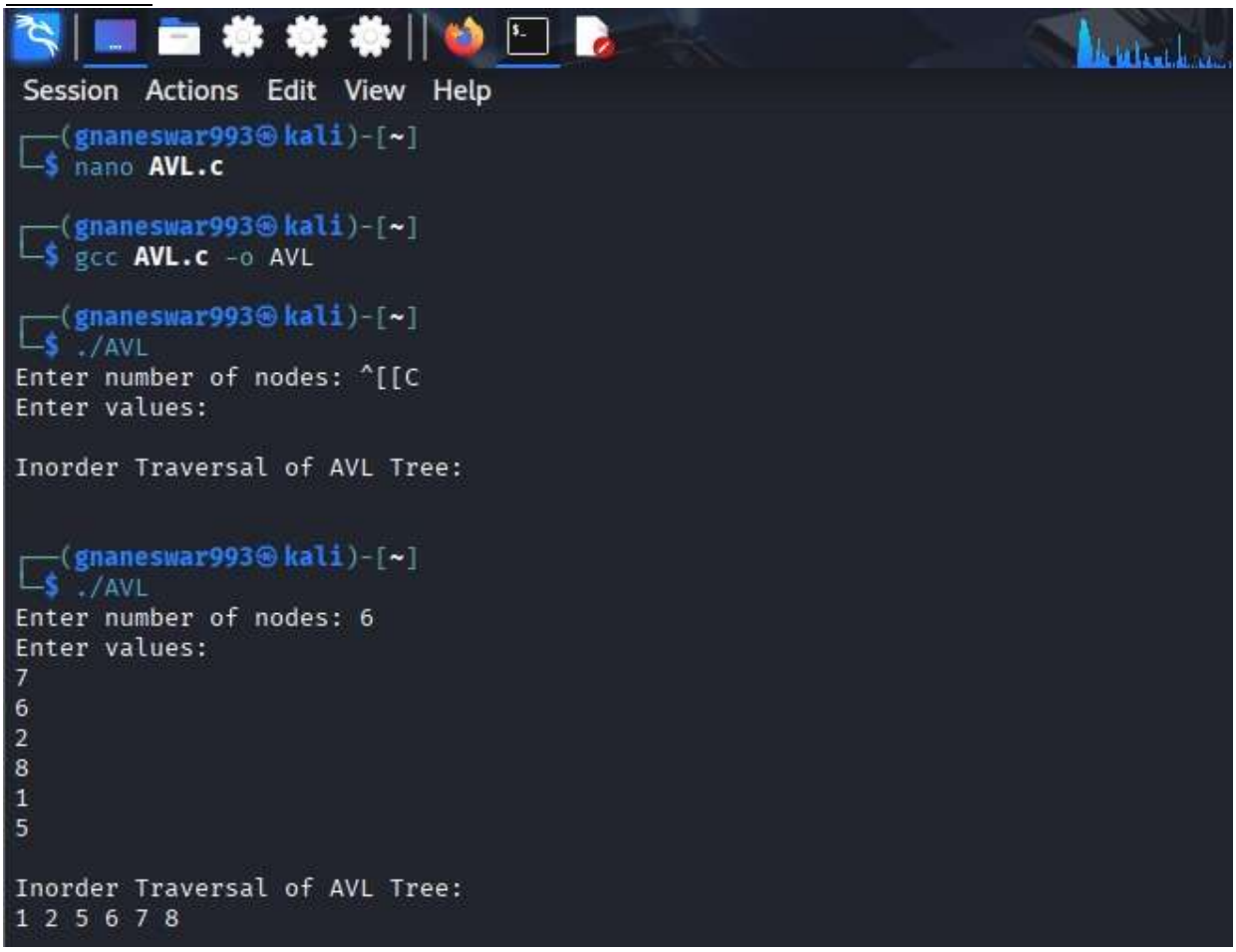
CODE: -

```
#include
<stdio.h>
#include
<stdlib.h>
struct
  Node {
  int key;
  struct Node
  *left; struct
  Node *right; int
  height;
};

int max(int a, int
  b) { return (a >
  b) ? a : b;
}

int height(struct Node
  *N) { if (N == NULL)
    return 0;
  return N-
  >height;
}

struct Node* newNode(int key) {
  struct Node* node = (struct
```

```
   Node*)malloc(sizeof(struct Node)); node->key =

   key;

   node->left =

   NULL; node-

   >right = NULL;

   node->height =

   1; return node;

}
struct Node *rightRotate(struct Node *y) {
struct Node *x = y->left;

   struct Node *T2 = x->right;
   x->right

   = y; y-

   >left =

   T2;

   y->height = max(height(y->left), height(y-

   >right)) + 1; x->height = max(height(x-

   >left), height(x->right)) + 1;

   return x;

}
struct Node *leftRotate(struct Node *x) {
   struct Node *y = x-

   >right; struct Node

   *T2 = y->left;

   y->left = x;

   x->right = T2;
   x->height = max(height(x->left), height(x-

   >right)) + 1; y->height = max(height(y-

   >left), height(y->right)) + 1;
```

```
  return y;
}
int getBalance(struct
  Node *N) { if (N ==
  NULL)
    return 0;
  return height(N->left) - height(N->right);
}
struct Node* insert(struct Node* node, int key) {
  if (node == NULL)
    return newNode(key);
  if (key < node->key)
    node->left = insert(node->left, key);
  else if (key > node->key)
    node->right = insert(node->right, key);
  else
    return node;
  node->height = 1 + max(height(node->left), height(node->right));
  int balance = getBalance(node);
  if (balance > 1 && key < node-
    >left->key) return
    rightRotate(node)
  if (balance < -1 && key > node-
    >right->key) return
    leftRotate(node)
  if (balance > 1 && key > node-
    >left->key) { node->left =
    leftRotate(node->left); return
    rightRotate(node);
  }
  if (balance < -1 && key < node-
```

```
    >right->key) { node->right =

    rightRotate(node->right); return

    leftRotate(node);

  }
  return node;

}
void inorder(struct Node *root) {
  if (root != NULL) {

    inorder(root->left);

    printf("%d ", root-

    >key);

    inorder(root-

    >right);

  }

}
int main() {
  struct Node *root =

  NULL; int n, value;

  printf("Enter number of nodes: ");

  scanf("%d", &n);

  printf("Enter values:\n");
  for(int i = 0; i < n;

    i++) { scanf("%d",

    &value);

  root = insert(root,

    value);


  }
  printf("\nInorder Traversal of AVL

  Tree:\n"); inorder(root);
```

```
  printf("\n");
  return 0;
}
```

OUTPUT: -